Professional Anomaly Detection Implementation Guide

Enterprise Data Feed Monitoring & Detection Framework

This document provides a professional implementation guide for anomaly detection in enterprise data feeds. The content is designed for developers, data engineers, architects, and technical teams responsible for monitoring data quality, data consistency, and operational integrity across analytical systems. The report explains: What anomaly detection is Types of anomalies How anomaly detection can be implemented in enterprise data pipelines Rule-based, statistical, and machine learning approaches Architecture recommendations SQL and Python implementation examples Monitoring and KPI recommendations

1. What is Anomaly Detection?

Anomaly detection is the process of identifying unexpected, unusual, invalid, suspicious, or inconsistent patterns in data. In enterprise data feeds, anomalies may indicate: Data quality problems Missing records Duplicate events Unexpected spikes or drops Incorrect values Fraudulent activities System failures Broken ETL pipelines Integration issues Example: If the average daily event count is 10,000 and suddenly drops to 100, this is considered an anomaly.

---

2. Types of Anomalies

There are three major categories of anomalies: **1. Point Anomaly**
A single data record behaves abnormally.

Example: Negative event duration Invalid country code Unexpected NULL value **2. Contextual Anomaly**
A value becomes abnormal only under specific conditions.

Example: High attendee count during weekends Unexpected event volume in a small region **3. Collective Anomaly**
A group of records together form an abnormal pattern.

Example: Continuous duplicate events Multiple events with identical timestamps Massive spike in updates

---

3. Your Data Feed Structure

The following columns are available in the feed and can be monitored for anomalies: event_id, event_country, therapeutic_area, product_name, indication_disease_state_name, product_id, indication_disease_state_id, department, event_start_date, event_end_date, event_name, event_location, event_status, event_type, event_owner, event_owner_id, event_sub_type, event_function, event_meeting_link, event_format, event_provider, event_source, event_source_type, lilly_person_id, eph_reference_id, eph_enterprise_id, attendee_type, attendee_status, attendee_status_updated_date, attended_topic, attendee_country, attendee_platform_type, tactic_id, platform_type, platform_id, institutional_authorization, customer_facing_user, created_date, updated_date, change_type.

---

4. Recommended Anomaly Detection Areas

Data Completeness Missing event_id NULL product_id Missing event_country **Uniqueness Validation** Duplicate event_id Duplicate eph_reference_id **Date Validations** event_end_date earlier than event_start_date Future dates Invalid timestamp patterns **Statistical Monitoring** Sudden increase in RowCount Unexpected StandardDeviation changes Entropy changes **Categorical Validation** Unexpected event_status values Invalid attendee_type Unknown platform_type **Relationship Validation** ColumnCorrelation changes Mismatch between country and platform

---

5. Available Metrics and Their Usage

The following metrics can be used for anomaly detection: **ColumnLength** – Detect unexpected text sizes.
ColumnValues – Detect invalid values.
Completeness – Identify NULL or missing data.
DistinctValuesCount – Monitor unexpected distinct count changes.
RowCount – Detect spikes or drops in data volume.
Uniqueness – Detect duplicates.
Mean – Detect statistical shifts.
Sum – Validate aggregated totals.
StandardDeviation – Identify volatility changes.
Entropy – Detect distribution changes.
UniqueValueRatio – Detect abnormal categorical patterns.
ColumnCount – Detect schema changes.
ColumnCorrelation – Detect broken relationships.
CustomSQL – Business-specific anomaly rules.
AllStatistic – Full profile-based anomaly monitoring.

---

6. Enterprise Architecture Recommendation

Recommended architecture: 1. Source Systems 2. Data Ingestion Layer 3. Raw Data Storage 4. Data Validation Layer 5. Anomaly Detection Engine 6. Alerting & Monitoring 7. Dashboard & Reporting Recommended technologies: Python SQL PySpark Kafka Airflow Databricks Snowflake Power BI

---

7. Rule-Based Detection Approach

Rule-based anomaly detection is the easiest and fastest implementation method. Example rules: event_id should never be NULL event_end_date must be greater than event_start_date event_country must belong to approved country list event_status should contain valid status values Advantages: Easy implementation Fast processing Good for operational monitoring

---

8. Statistical Detection Approach

Statistical anomaly detection uses historical distributions. Techniques: Z-score detection Percentile analysis Standard deviation thresholds Moving averages Time-series trend analysis Example: If daily RowCount exceeds 3 standard deviations from average, raise anomaly alert.

---

9. Machine Learning Detection Approach

Machine learning can identify hidden patterns and complex anomalies. Recommended models: Isolation Forest Local Outlier Factor Autoencoders One-Class SVM DBSCAN Use cases: Behavioral anomalies Large-scale event monitoring Unknown anomaly discovery Pattern drift detection

---

10. SQL Implementation Examples

Duplicate Detection SELECT event_id, COUNT(*) FROM events GROUP BY event_id HAVING COUNT(*) > 1;

Date Validation SELECT * FROM events WHERE event_end_date < event_start_date;

Missing Values SELECT * FROM events WHERE event_country IS NULL;

Volume Monitoring SELECT created_date, COUNT(*) AS total_events FROM events GROUP BY created_date;

---

11. Python Anomaly Detection Example

Example implementation using Isolation Forest: 1. Load event data 2. Select numerical/statistical features 3. Train anomaly model 4. Predict anomaly scores 5. Store anomaly results 6. Trigger alerts Recommended libraries: Pandas Scikit-learn PySpark ML Numpy

---

12. Real-Time Monitoring Recommendation

Real-time anomaly monitoring can be implemented using: Kafka Streams Spark Streaming Flink Airflow Cloud-native event services Recommended monitoring: Hourly RowCount monitoring Schema change detection Real-time duplicate validation Latency tracking Data freshness validation

---

13. KPI Dashboard Recommendations

Recommended dashboard KPIs: Total Records Processed Total Failed Records Duplicate Percentage Missing Value Percentage Anomaly Count by Country Data Freshness Delay Daily Event Volume Trend Platform-wise Error Rate Schema Change Alerts

---

14. Best Practices

Implement multi-layer anomaly detection Combine rule-based and ML-based detection Store anomaly history Enable automated alerting Maintain threshold configuration tables Use metadata-driven validations Track schema evolution Create SLA-based monitoring Enable audit logging

---

15. Recommended Enterprise Roadmap

Phase 1: Basic data quality validation. Phase 2: Statistical anomaly monitoring. Phase 3: Real-time anomaly detection. Phase 4: Machine learning anomaly prediction. Phase 5: Self-healing pipelines and automated remediation.

Disclaimer

This document is provided for general information and educational purposes only. It does not constitute professional, legal, financial, medical, or regulatory advice. Implementation approaches should be reviewed and validated according to organizational standards, security policies, governance requirements, and production architecture constraints.

---

16. Recommended Validation Matrix

Validation Area	Columns	Recommended Metric
Duplicate Detection	event_id, eph_reference_id	Uniqueness
Missing Values	event_country, product_id	Completeness
Volume Monitoring	Daily Records	RowCount
Statistical Drift	Event Volume	Mean, StandardDeviation
Distribution Changes	event_status	Entropy
Schema Validation	All Columns	ColumnCount
Business Rule Checks	Dates, IDs	CustomSQL